

SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

Team 34: The Hacktivist

System Architecture & Design (SAD)

Documentation: Silver Finn

1. Introduction

This section introduces **Silver Finn**, an intelligent workshop management platform designed for the **UMHack** competition. It provides a strategic solution to modernising automotive workshop operations through immersive 3D technology and AI-driven insights.

1.1 Purpose

The system analysis document highlights the technical scope and design decisions behind the Silver Finn development. It serves as a blueprint for transforming a static 3D mockup into a functional, multi-tenant enterprise application.

The key elements of the system covered are:

- **Architecture:** A client-server web application optimised for iPad Safari (16:9 aspect ratio).
- **Data Flows:** The progression of service data from 3D hotspot interaction to PostgreSQL persistence and final PDF report generation.
- **Model Process:** The end-to-end workflow of a guided vehicle inspection, from pulsing amber hotspots to a finalised service history.
- **Role of Reference:** This document guides developers in implementing the six-phase build plan and ensures technical alignment with the original mockup architecture.

1.2 Background

Traditional workshops often rely on manual checklists and fragmented history. Silver Finn bridges this gap by integrating a high-end, Asphalt-style 3D car viewer with proactive maintenance alerts.

- **Previous Version:** The project originates from a React mockup utilizing **React Three Fiber** for a Mercedes C63 AMG model viewer.
- **Major Architectural Changes:** Transitioning from a frontend-only mockup to a full-stack system with a Node.js backend, PostgreSQL database, and ChromaDB vector store.
- **New Capabilities:** Introduction of RAG-powered AI chat for OEM specs and a statistical prediction engine for parts replacement.

1.3 Target Stakeholders

Stakeholders	Roles	Expectations
Mechanic	Performs inspections and logs data via the 3D interface.	Intuitive guided flow, easy photo uploads, and quick access to torque/spec data.
Service Advisor	Manages customer profiles and generates service reports.	Clear vehicle timelines, automated PDF generation, and proactive alerts.
Admin	Oversees workshop settings and user management.	Multi-tenant support to manage different workshop branches.
Dev Team	Build and maintain the platform using React and Node.js.	Adherence to the established Prisma schema and modular repository structure.

2. System Architecture & Design

2.1 High Level Architecture

2.1.1 Overview

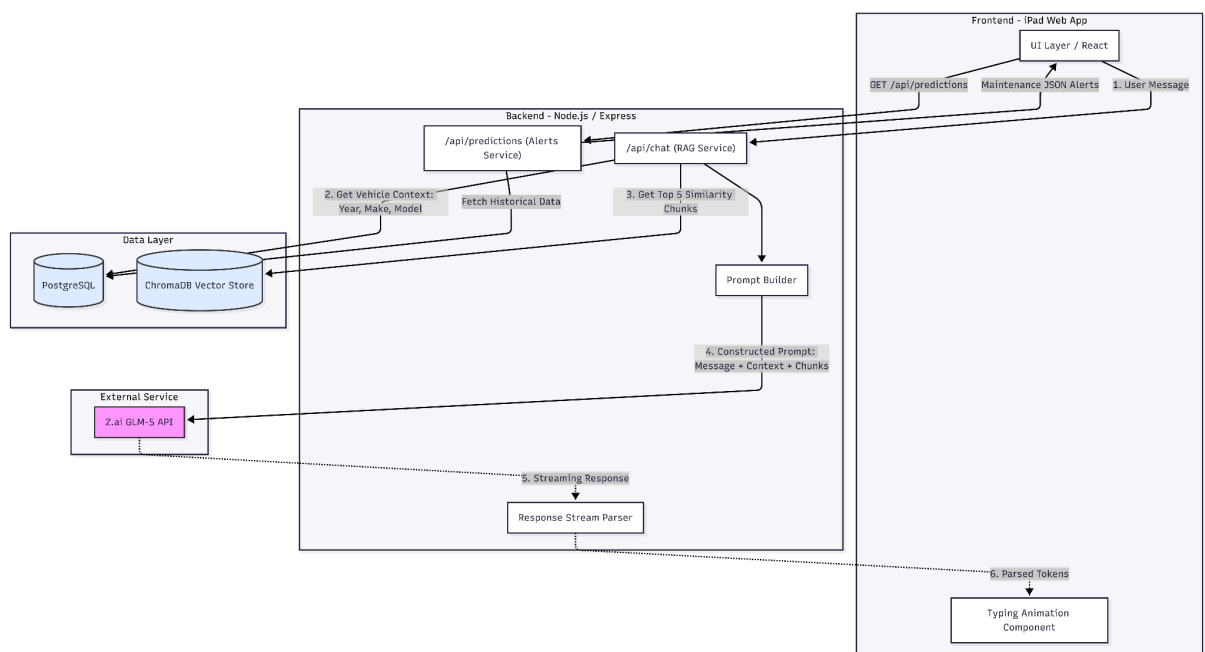
Type	Details
System	iPad-optimised Web Application (Safari).
Architecture	Client-Server with RAG (Retrieval-Augmented Generation) Service Layer.

Silver Finn is structured as a layered system where a **React 19** frontend interacts with a **Node.js/Express** backend. The frontend manages the 3D rendering context (Layer 0) and the UI overlay (Layer 1). The backend facilitates persistent storage via **PostgreSQL** and specialised AI reasoning via the **Gemini API**.

2.1.2 LLM as Service Layer

The **Gemini** model is integrated as a core reasoning engine. It does not just provide generic chat; it is grounded in a specific car knowledge base containing Malaysian car models like Proton and Perodua.

2.1.3 Dependency Diagram



2.2.2 Sequence Diagram: Service Checklist Flow

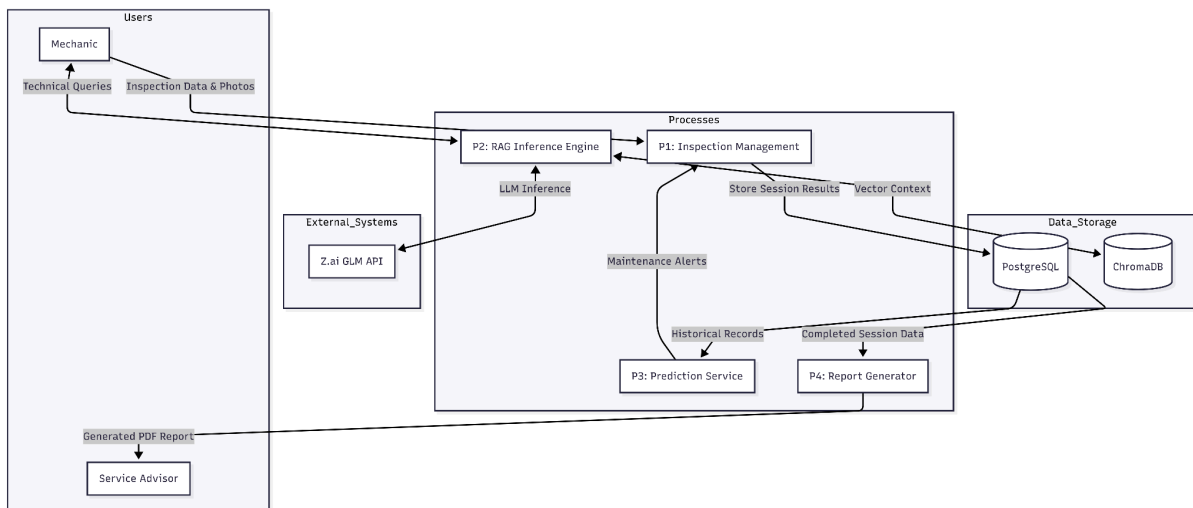
1. **Start:** The mechanic opens a vehicle record; the backend returns the appropriate mileage-based checklist template.
2. **Interaction:** Mechanic taps a pulsing hotspot (e.g., Engine); the UI displays a **CheckItem** card.
3. **Data Entry:** Result (PASS/FAIL) and photos are sent to the backend and stored in Cloudinary/PostgreSQL.
4. **Completion:** Once all 8 zones are completed, the UI triggers a PDF export request.

2.2 Technological Stack

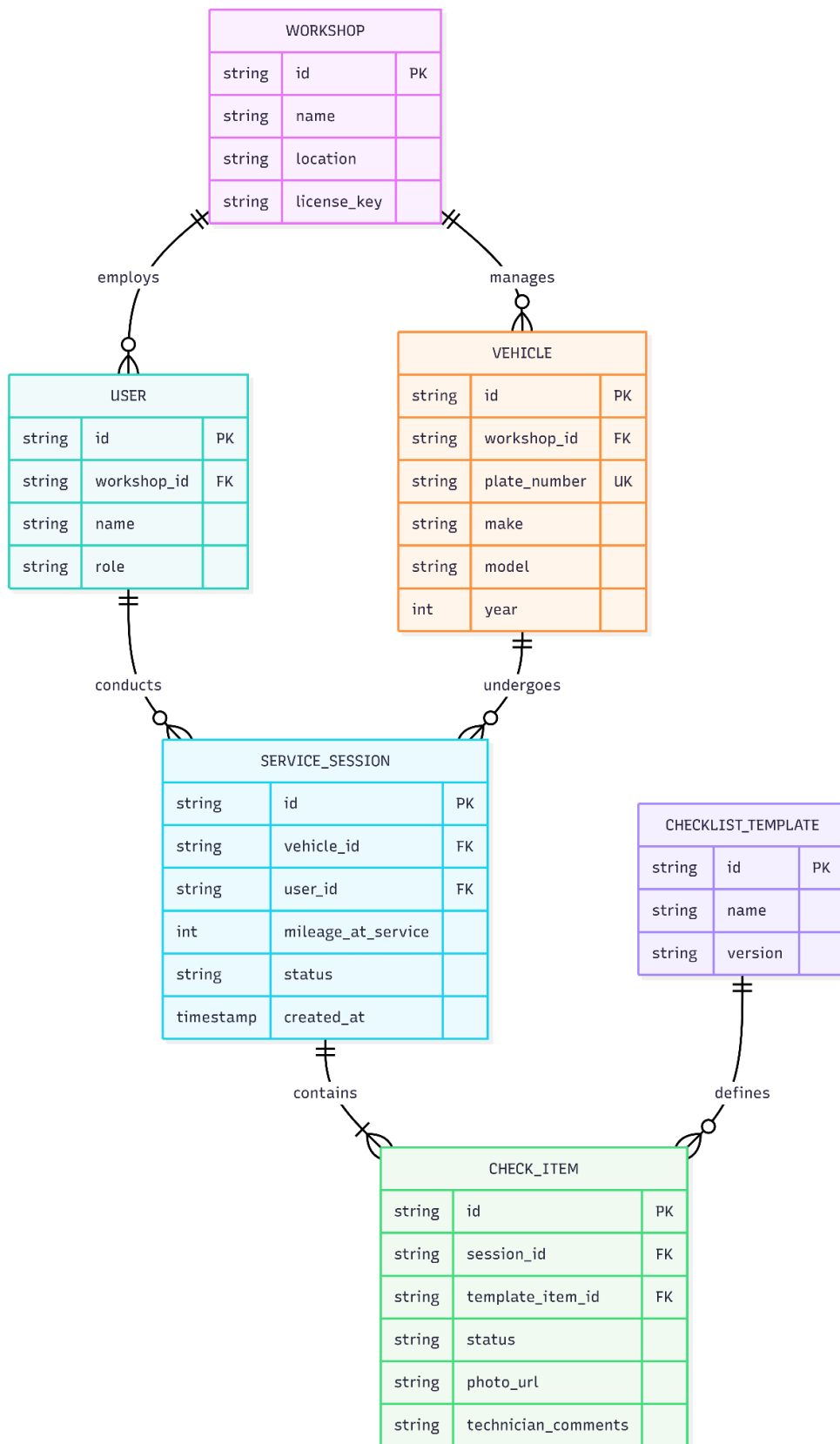
- **Frontend:** React 19 + Vite 8 + Tailwind CSS (Dark Metallic Theme).
- **3D Engine:** Three.js + React Three Fiber + Drei.
- **Backend:** Node.js + Express.
- **Database:** PostgreSQL on Neon (Relational) + ChromaDB (Vector).
- **Cloud/Hosting:** Vercel (Frontend), Railway (Backend), Cloudinary (Images).

2.3 Key Data Flows

2.3.1 Data Flow Diagram (DFD)



2.3.2 Normalised Database Schema (ERD)



3. Functional Requirements & Scope

3.1 Minimum Viable Product (MVP)

#	Features	Description
1	Guided 3D Checklist	Interactive 3D hotspots guide mechanics through 8 service zones with PASS/FAIL logging.
2	RAG Chat Assistant	A side-panel assistant providing technical specs and torque values via Gemini.
3	Prediction Engine	Statistical flagging of parts likely to fail based on historical data of similar models.
4	Service History	A reverse-chronological timeline of past vehicle sessions and costs.
5	PDF Report Export	Client-side generation of a professional service summary for the customer.

3.2 Non-Functional Requirements (NFRs)

Quality	Requirements	Implementation
Usability	The interface must be functional on iPad Safari.	16:9 fixed layout with absolute-positioned UI overlays.
Portability	Dark metallic theme for high-contrast workshop environments.	CSS variables for background (#0A0C0F) and amber accents.

Cost Efficiency	Must operate within free hosting limits for the hackathon.	Utilizing Neon's 10GB free tier and Railway's \$5 credit.
AI Reliability	Prediction logic must be performant with 750+ records.	Use simple statistical frequency counts rather than heavy ML models.

4. Monitor, Evaluation, Assumptions & Dependencies

4.1 Technical Evaluation

- **Priority Matrix:**
 - **P1 (Critical):** 3D model fails to load or hotspot markers are misaligned, blocking the checklist.
 - **P2 (High):** API failure for RAG chat; the system should degrade to show static service charts.
 - **P3 (Low):** PDF styling issues or minor prediction inaccuracies.

4.3 Assumptions

- **Market:** Primary focus on Malaysian automotive brands (Proton, Perodua, Toyota, Honda).
- **Hardware:** Mechanics have stable internet and access to iPad devices.
- **Data:** The generated 750-1000 "fake" records will sufficiently demo the prediction logic.

4.4 External Dependencies

Tools	Purpose	Risks
Gemini API	Core LLM for RAG and reasoning.	API downtime or key expiration.
Neon PostgreSQL	Primary database hosting.	Storage limits on the free tier (10GB).
Cloudinary	Storage for inspection photos.	Bandwidth limits (25GB) during heavy demo use.

5. Project Management & Team Contributions

5.1 Project Timeline

- **Phase 1-2 (Backend & Theme):** Scaffold Express, Prisma schema, and apply Dark Metallic UI.
- **Phase 3-4 (3D & History):** Map 8 hotspots for guided mode and build the timeline view.
- **Phase 5-6 (AI & Predictions):** Ingest knowledge base into ChromaDB and finalize prediction logic.

5.2 Team Members Role

- **Dhaneesh (Frontend Developer):** Full-stack integration, Three.js hotspot remapping, and Prisma schema design.
- **Syed Azhar (AI/Backend Developer):** Gemini API integration, RAG service, and statistical prediction algorithm.
- **Elvira (UI/UX):** Tailwind CSS dark theme implementation and Framer Motion animations.

5.3 Recommendations

- **Scalability:** Use **Redis** for caching frequent RAG queries to save on API token costs.
- **Stability:** Implement **Zustand** or **Redux** for managing the "active session" state to prevent data loss if the iPad browser refreshes.